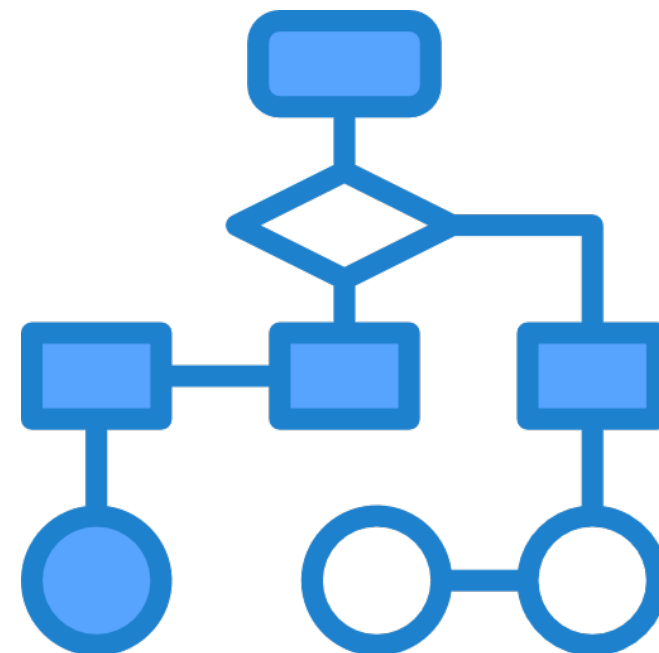


Programmazione Web e Mobile con Laboratorio

Lezione 10 - Basi di JavaScript 2
(Strutture di controllo, oggetti, eventi)

Strutture di controllo

- Gestiscono il flusso di esecuzione del codice e consentono decisioni logiche o ripetizioni
- if, else if, else
- switch
- for, for...in, for...of
- while, do...while



if, else if, else

- Eseguono blocchi di codice in base a condizioni specifiche

JS

```
let x = 10;  
if(x > 5) {  
    console.log("x è maggiore di 5");  
}  
else if(x > 2) {  
    console.log("x è compreso tra 3 e 5");  
}  
else {  
    console.log("x è 2 o inferiore");  
}
```

switch

- Alternativa ad una serie di if...else per valutare più valori di una stessa variabile

JS

```
let giorno = "Lunedì";
switch (giorno) {
  case "Lunedì":
    console.log("Inizio delle lezioni");
    break;
  case "Venerdì":
    console.log("Fine delle lezioni");
    break;
  default:
    console.log("Giorno ordinario");
}
```

for, for...in, for...of

- **for**: esegue un blocco di codice un numero specifico di volte

JS

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

- **for...in**: itera sulle proprietà di un oggetto

JS

```
let persona = { nome: "Luca", età: 25 };  
for (let chiave in persona) {  
  console.log(chiave, persona[chave]);  
}
```

- **for...of**: itera su array o altre strutture **iterable**

JS

```
let arr = [1, 2, 3];  
for (let numero of arr) {  
  console.log(numero);  
}
```

while, do...while

- **while:** esegue il blocco di codice finché la condizione specificata è vera

JS

```
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

- **do...while:** esegue il codice almeno una volta

JS

```
let i = 8;
do {
  console.log(i);
  i++;
}
while (i < 5);
```

Esercizio

- Creare una funzione js che legge il valore N da un input di tipo number e inserisce nella pagina N paragrafi di testo

Inserisci paragrafi

Usa la casella di input per specificare il numero di paragrafi da aggiungere

Paragrafo #1

Paragrafo #2

Paragrafo #3

Paragrafo #4

Paragrafo #5

Paragrafo #6

Array

Array in JavaScript

- Struttura di dati che permette di memorizzare una sequenza di valori (numeri, stringhe, oggetti e altri array)
- Gli array sono oggetti
- Ogni valore in un array ha un indice numerico (a partire da 0)
- Possono contenere valori dello stesso tipo o di tipi diversi
- Gli array sono dinamici, quindi possono crescere o ridursi durante l'esecuzione del programma

Creazione di un array

- Sintassi letterale

```
JS let array1 = [1, 2, 3, 4, 5];
```

- Con costruttore

```
JS let array2 = new Array("apple", 1, true, null);
```

- Vuoto con una lunghezza predefinita

```
JS let array3 = new Array(10);
```

Accesso agli elementi

- Accedere al primo elemento

```
JS let el = array[0];
```

- Accedere all'ultimo elemento

```
JS let el = array[array.length - 1]
```

La proprietà .length
indica il numero di
elementi di un array

- Modificare un elemento

```
JS let el = array[indice] = newVal
```

Metodi utili - manipolazione

- **push(el)**: aggiunge uno o più elementi alla fine dell'array



Metodi utili - manipolazione

- **push(el)**: aggiunge uno o più elementi alla fine dell'array



Metodi utili - manipolazione

- **push(el)**: aggiunge uno o più elementi alla fine dell'array
- **pop()**: rimuove e restituisce l'ultimo elemento dell'array



Metodi utili - manipolazione

- **push(el)**: aggiunge uno o più elementi alla fine dell'array
- **pop()**: rimuove e restituisce l'ultimo elemento dell'array
- **unshift(el)**: aggiunge uno o più elementi all'inizio dell'array



Metodi utili - manipolazione

- **push(el)**: aggiunge uno o più elementi alla fine dell'array
- **pop()**: rimuove e restituisce l'ultimo elemento dell'array
- **unshift(el)**: aggiunge uno o più elementi all'inizio dell'array
- **shift()**: rimuove e restituisce il primo elemento dell'array



Metodi utili - ricerca

- **indexOf(el)**: restituisce l'indice della prima occorrenza di un elemento (o -1 se non presente)
- **includes(el)**: restituisce true se l'elemento esiste nell'array
- **find(callback)**: restituisce il primo elemento che soddisfa la condizione definita nella callback (altrimenti undefined)

JS

```
let i = arr.indexOf("C") // 2  
let check = arr.includes("B") // true  
let risultato = arr.find(el => el > "C"); // D
```

A

B

C

D

C

Metodi utili - ordinamento

- **sort()**: ordina gli elementi in ordine alfabetico o personalizzato con una funzione di confronto
- **reverse()**: inverte l'ordine degli elementi dell'array

JS

```
let arr = ["L","E","T","T","E","R","E"];  
arr.sort() // ['E', 'E', 'E', 'L', 'R', 'T', 'T']  
arr.reverse(); // ['T', 'T', 'R', 'L', 'E', 'E', 'E']
```

Metodi utili - ordinamento

- **sort()**: ordina gli elementi in ordine alfabetico o personalizzato con una funzione di confronto
- **reverse()**: inverte l'ordine degli elementi dell'array

JS

```
let arr = ["L","E","T","T","E","R","E"];  
arr.sort() // ['E', 'E', 'E', 'L', 'R', 'T', 'T']  
arr.reverse(); // ['T', 'T', 'R', 'L', 'E', 'E', 'E']
```

JS

```
let arr2 = [2,4,1,6,4,5,5,9,22,0,3,-8,14,7];  
arr2.sort() // [-8, 0, 1, 14, 2, 22, 3, 4, 4, 5, 5, 6, 7, 9]
```

Metodi utili - ordinamento

- **sort()**: ordina gli elementi in ordine alfabetico o personalizzato con una funzione di confronto
- **reverse()**: inverte l'ordine degli elementi dell'array

JS

```
let arr = ["L","E","T","T","E","R","E"];  
arr.sort() // ['E', 'E', 'E', 'L', 'R', 'T', 'T']  
arr.reverse(); // ['T', 'T', 'R', 'L', 'E', 'E', 'E']
```

JS

```
let arr2 = [2,4,1,6,4,5,5,9,22,0,3,-8,14,7];  
arr2.sort() // [-8, 0, 1, 14, 2, 22, 3, 4, 4, 5, 5, 6, 7, 9]  
arr2.sort((a, b) => a - b); // [-8, 0, 1, 2, 3, 4, 4, 5, 5, 6, 7, 9, 14, 22]
```

Metodi utili - iterazione

- **forEach(callback):** esegue una funzione per ogni elemento

JS

```
const frutta = ["mela", "banana", "arancia"];  
frutta.forEach((frutto) => {console.log(frutto);});
```

Metodi utili - iterazione

- **forEach(callback):** esegue una funzione per ogni elemento

JS

```
const frutta = ["mela", "banana", "arancia"];  
frutta.forEach((frutto) => {console.log(frutto);});
```

- **map(callback):** crea un nuovo array con il risultato della callback su ogni elemento

JS

```
const fruttaMaiuscola = frutta.map((frutto) => {  
    return frutto.toUpperCase();  
});
```

Metodi utili - iterazione

- **forEach(callback):** esegue una funzione per ogni elemento

JS

```
const frutta = ["mela", "banana", "arancia"];  
frutta.forEach((frutto) => {console.log(frutto);});
```

- **map(callback):** crea un nuovo array con il risultato della callback su ogni elemento

JS

```
const fruttaMaiuscola = frutta.map((frutto) => {  
    return frutto.toUpperCase();  
});
```

- **filter(callback):** crea un nuovo array con tutti gli elementi che soddisfano la condizione specificata nella callback

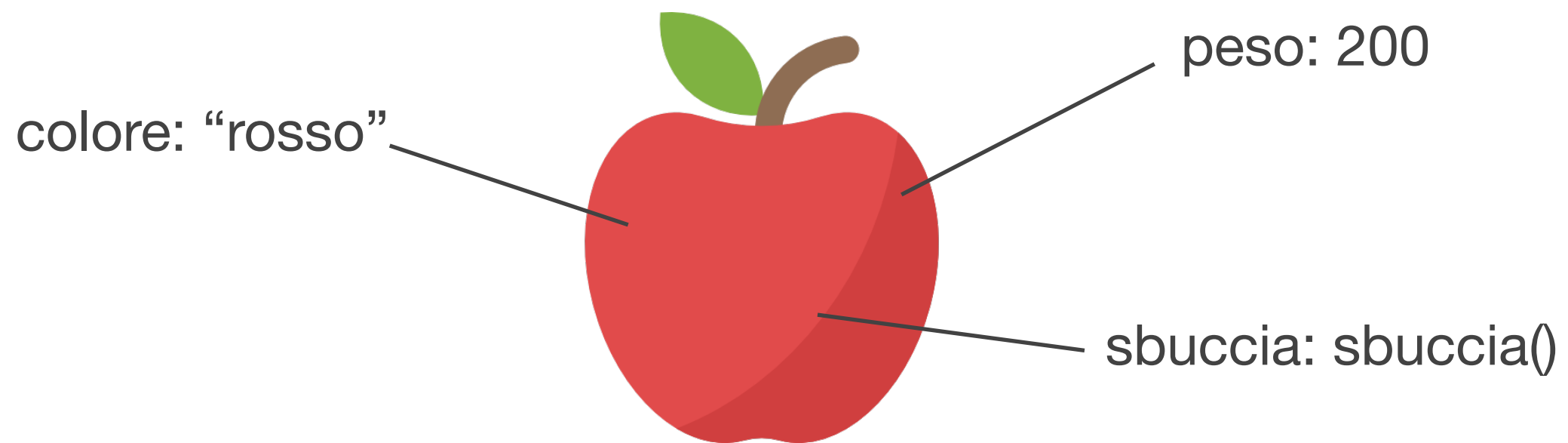
JS

```
const numeri = [5, 12, 8, 130, 44];  
const numeriGrandi = numeri.filter((numero) => numero > 10);
```


Oggetti

Oggetti in JavaScript

- Collezioni di proprietà identificate da coppie chiave-valore
- Gli oggetti usano come indici i nomi delle proprietà
- Gli array sono oggetti con indici numerici



Creazione di un oggetto

- Notazione letterale

JS

```
let persona = {  
  nome: "Lorena",  
  età: 30,  
  occupazione: "Insegnante"  
};
```

- Con il costruttore Object()

JS

```
let persona = new Object();  
persona.nome = "Lorena";  
persona.età = 30;  
persona.occupazione = "Insegnante";
```

Accesso alle proprietà

- Notazione con parentesi quadre (utilizzata per gli array o quando la chiave è dinamica)

```
JS let nome = persona["nome"];
```

- Notazione a punto

```
JS let nome = persona.nome;
```

- Aggiunta, modifica e rimozione

```
JS persona.indirizzo = "Via Roma 1";  
persona["email"] = "marco@example.com";  
delete persona.occupazione;
```

Metodi

- sono funzioni associate ad un oggetto

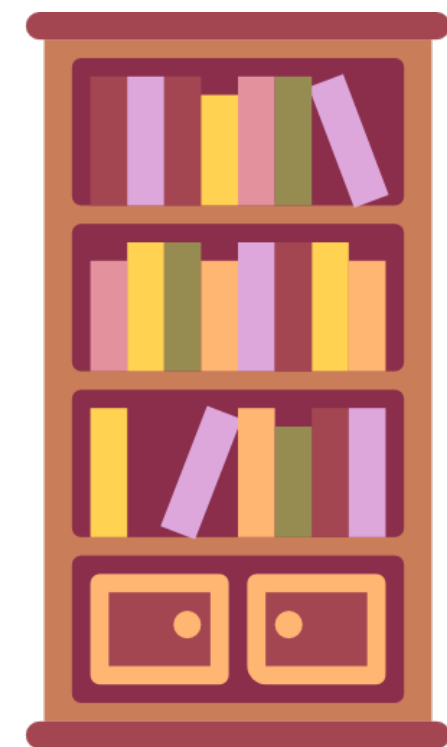
JS

```
let auto = {  
  marca: "Fiat",  
  modello: "Panda",  
  anno: 2020,  
  saluta: function() {  
    console.log("Ciao, sono una " + this.marca + " " + this.modello);  
  }  
};
```

- **this** fa riferimento all'oggetto stesso e permette di accedere alle sue proprietà

Esercizio

- Creare e manipolare un catalogo di libri in JS e HTML
- Funzioni da implementare:
 - `aggiungiLibro(titolo, autore, anno)`
 - `mostraCatalogo()`
 - `trovaLibroPerTitolo(titolo)`
 - `rimuoviLibro(titolo)`
 - `ordinaLibriPerAnno()`



Eventi

Eventi in JavaScript

- Sono azioni che si verificano all'interno del browser
- Rispondono alle interazioni dell'utente
- Tipi comuni:
 - click, mouseover, mouseout
 - keydown, keyup
 - submit, focus
 - load, resize

Aggiungere eventi

- Tre approcci possibili:

1. HTML inline: mescola HTML e JS, un solo gestore per evento

```
HTML <button onclick="alert('Ciao!')">Clicca</button>
```

2. Proprietà JS: separa HTML e JS, un solo gestore per evento

```
JS bottone.onclick = function() { alert("Ciao!"); };
```

3. Listener: più codice, ma più gestori per evento (accodati)

```
JS bottone.addEventListener("click", function() { alert("Ciao!"); });
```


Aggiungere eventi

- Tre approcci possibili:

1. HTML inline: mescola HTML e JS, un solo gestore per evento

```
HTML <button onclick="alert('Ciao!')">Clicca</button>
```

2. Proprietà JS: separa HTML e JS, un solo gestore per evento

```
JS bottone.onclick = function() { alert("Ciao!"); };
```

3. Listener: più codice, ma più gestori per evento (accodati)

```
JS bottone.addEventListener("click", function() { alert("Ciao!"); });
```

Proprietà evento

Eventi del mouse

Proprietà	Evento
<code>.onclick</code>	Click con il tasto sinistro del mouse
<code>.ondblclick</code>	Doppio click
<code>.onmousedown</code>	Tasto del mouse premuto
<code>.onmouseup</code>	Tasto del mouse rilasciato
<code>.onmouseover</code>	Mouse entra in elemento o suo figlio
<code>.onmouseout</code>	Mouse esce da elemento o suo figlio
<code>.onmouseenter</code>	Mouse entra in elemento
<code>.onmouseleave</code>	Mouse esce da elemento
<code>.onmousemove</code>	Movimento del mouse sull'elemento

Eventi da tastiera

Proprietà	Evento
<code>.onkeydown</code>	Tasto premuto
<code>.onkeyup</code>	Tasto rilasciato

Eventi delle form

Proprietà	Evento
<code>.onsubmit</code>	Invio di una form
<code>.onreset</code>	Reset di una form
<code>.onchange</code>	A modifica di un campo avvenuta
<code>.oninput</code>	Ogni volta che l'input cambia
<code>.onfocus</code>	Elemento messo a fuoco
<code>.onblur</code>	Perdita del focus

Eventi della finestra

Proprietà	Evento
<code>.onload</code>	La pagina o un'immagine è caricata
<code>.onresize</code>	Cambio di dimensione della finestra
<code>.onscroll</code>	Scroll della finestra o di un div

window.onload

- Il codice JS viene eseguito immediatamente al caricamento dello script
- Se il DOM non è caricato completamente, lo script potrebbe non avere accesso ad alcuni elementi della pagina
- Soluzione: usare **window.onload**

JS

```
window.onload = function() {  
    console.log("La pagina e tutte le risorse sono state caricate!");  
};
```

Listener evento

Eventi del mouse

Proprietà	Evento
<code>click</code>	Click con tasto sinistro
<code>dblclick</code>	Doppio click
<code>mousedown</code>	Tasto mouse premuto
<code>mouseup</code>	Tasto mouse rilasciato
<code>mousemove</code>	Movimento del mouse sull'elemento
<code>mouseover</code>	Mouse entra in elemento o suo figlio
<code>mouseout</code>	Mouse esce da elemento o suo figlio
<code>mouseenter</code>	Mouse entra in elemento
<code>mouseleave</code>	Mouse esce da elemento
<code>contextmenu</code>	Click destro

Eventi da tastiera

Proprietà	Evento
<code>keydown</code>	Tasto premuto
<code>keyup</code>	Tasto rilasciato

Eventi delle form

Proprietà	Evento
<code>submit</code>	Invio di una form
<code>reset</code>	Reset di una form
<code>change</code>	A modifica di un campo avvenuta
<code>input</code>	Ogni volta che l'input cambia
<code>focus</code>	Elemento messo a fuoco
<code>blur</code>	Perdita del focus
<code>focusin</code>	Focus su un figlio
<code>focusout</code>	Uscita del focus da un figlio

Eventi della finestra

Proprietà	Evento
<code>load</code>	La pagina o un'immagine è caricata
<code>beforeunload</code>	Prima di uscire dalla pagina
<code>resize</code>	Cambio di dimensione della finestra
<code>scroll</code>	Scroll della finestra o di un div
<code>error</code>	Errore durante il caricamento

preventDefault()

- Impedisce il comportamento predefinito di un evento

JS

```
document.querySelector("form").addEventListener("submit", function(evento) {  
    evento.preventDefault();  
    console.log("Invio bloccato. Qui puoi gestire i dati.");  
});
```

- Esempi:
 - Evita che una form ricarichi la pagina all'invio
 - Impedisce la navigazione verso un URL quando si clicca su un'ancora
 - Blocca il menu standard del browser al click destro
 - Inibisce tasti speciali come backspace, F5 e tab

Esercizio

- Creare una form HTML con una textarea
- Aggiungere un evento di submit che intercetti l'invio del form
- Se il testo contiene una "ban word", impedire l'invio e mostrare un alert

Scrivi un testo con controllo delle parole vietate

Lista di parole vietate: casa, acqua, gennaio

Il mio mese preferito è Gennaio!

Invia